

# dsABRE: A SABRE-Style Router for Multi-Core Distributed Quantum Computers

Sanjiang Li \*

**Abstract**—A distributed quantum computer (DQC) runs a circuit across several small cores linked by entanglement. Executing a gate whose operands sit in different cores consumes Einstein–Podolsky–Rosen (EPR) pairs, orders of magnitude costlier than a local gate and so the quantity a distributed router must minimise. On a single chip that role falls to SABRE, the state of the art for monolithic layout and routing and Qiskit’s default, which scores a SWAP by the front-layer distance it saves plus a bounded lookahead. We present dsABRE, which carries that objective across cores, scoring each inter-core move by one rule: the resource it spends, plus the capacity it consumes, plus the progress it buys in one *routing potential*—front-layer distance plus a lookahead discounted by depth in the circuit’s directed acyclic graph (DAG). Its added terms are not independent heuristics but the two respects in which a teleport is unlike a SWAP: it fires only at a communication port, and it consumes a free slot in the core it lands in. Across 21 Munich Quantum Toolkit (MQT) Bench circuits at 25–64 logical qubits dsABRE cuts geometric-mean EPR consumption by 49–62% against TelesABRE, and routes all four of this paper’s 27 circuits on which TelesABRE does not converge—including a 360-qubit QFT on twenty cores, compiled in 48 s; a capacity condition, checked once before routing, further guarantees dsABRE itself never aborts. Code is at <https://github.com/ebony72/dsabre>.

**Index Terms**—distributed quantum computing, circuit routing, qubit teleportation, SABRE, EPR minimisation, qubit mapping, lookahead heuristic

## I. INTRODUCTION

The near-term path to fault-tolerant quantum computing requires processors with thousands of high-fidelity qubits. No single monolithic chip can economically deliver this scale today. Instead, the field is converging on *distributed quantum computers* (DQC) [1]: multiple small quantum processing units (QPUs, also called cores) interconnected by photonic or microwave links that support remote entanglement generation.

Executing a quantum circuit on a DQC requires a *distributed compilation* pipeline that, like its monolithic counterpart, decomposes into three problem classes [2]. (i) *Qubit allocation* statically assigns logical qubits to cores, typically by hypergraph partitioning [3], [4]. (ii) *Communication-primitive selection* decides how each non-local two-qubit gate is realised—by bringing both qubits to the same core, or by executing the gate remotely [5], [6] (Section II-B describes both primitives and their costs). (iii) *Routing and scheduling* orders the resulting intra-core SWAPs, teleports, and entanglement-generation requests in time, subject to finite entanglement rates and to the capacity of the *communication ports*—the physical

qubits that terminate an inter-core link, of which each core has finitely many [7], [8]. All three are judged in one currency: every primitive consumes EPR pairs, whose generation is slow, noisy and rate-limited in current hardware [9], [10]—orders of magnitude costlier than a local gate—so EPR consumption is the dominant objective of distributed compilation, and the one this paper minimises.

**Two families.** Compilers that solve (i)–(iii) differ less in their optimisation machinery than in what they assume the machine can do. The first family reduces distribution to (*hyper*)*graph partitioning*, cutting a hypergraph whose hyperedges collect the gates that can share one entangled pair, so that the cut weight is the number of EPR pairs consumed [3], [4]. The appeal is a global, one-shot optimisation backed by mature solvers; the cost is an idealised machine, in which intra-core routing is free, the number of links joining two cores goes unrecorded, per-core communication memory is effectively unbounded, and an entangled pair survives its whole hyperedge. These are claims about the machine rather than bookkeeping conventions, and they set the counts: giving the same partitioner this device’s own link and slot budgets is enough to reverse which tool wins at 64 qubits (Section IV-E).

The second family gives up the global view and decides during routing, pricing those same constraints into each candidate move instead of idealising them away. Its monolithic ancestor is SABRE [11], Qiskit’s default layout and routing pass (Section II-D): at each step it inserts the SWAP that most reduces the distance between the operands of the gates waiting to execute, with a discounted look at the gates behind them. Two recent routers carry that score across cores. TelesABRE [12] prices an inter-core move by a Dijkstra computation on a contracted communication graph; DMapS [13] pairs a hypergraph partitioner with a SABRE-derived router whose only cross-core primitive is a two-EPR *remote SWAP*.

What neither prices is the free capacity of the core a move lands in. A remote SWAP exchanges two qubits and leaves every core’s occupancy unchanged, so DMapS never faces the question—and pays two EPR pairs for each crossing. TelesABRE pays one, and meets the resulting pressure defensively: a fixed penalty once a core is already nearly full, and a safety valve that drops most of the lookahead once routing stalls, rather than a signal that grades continuously with how much slack a landing core has left. Dense or large instances therefore either burn additional EPR pairs or fail to converge.

The defect is not specific to one penalty: porting a SWAP-based router to a DQC is not a drop-in substitution of teleport for SWAP. A SWAP is symmetric and leaves the chip’s occupied-slot count unchanged. A teleport is one-directional;

S. Li is with the Centre for Quantum Software and Information, University of Technology Sydney, Australia (e-mail: [sanjiang.li@uts.edu.au](mailto:sanjiang.li@uts.edu.au)).

\*ORCID: 0000-0002-3332-2546.

it fires only at a communication port, so it is ready only once its qubit has been walked there and whatever occupies the port has been evicted; and where it lands, the destination core is left more crowded than before.

This paper presents dSABRE, whose answer is a single rule rather than a collection of fixes: one score for every inter-core move, adding to SABRE’s distance-plus-lookahead objective exactly two terms, one per departure—the staging that *availability* demands, and the *capacity* the move consumes (Section III-A). Everything else in Section III is one of those two terms, the lookahead construction they are scored against (Section III-E), or the recovery for the one state in which a distance objective stops being a usable surrogate—the cores an action needs are saturated (Section III-F). dSABRE moves qubits by teledata alone, which is what makes its counts unconditional: a pair is consumed by the teleport that creates it, so no score models entanglement lifetime and every count we report holds under any bound on it (Section IV-E).

### Contributions.

- **A capacity-aware macro-action teleportation score.** dSABRE scores the whole macro-action—the staging SWAPs, any eviction, and the teleport itself—as one move, carrying SABRE’s front-layer distance change and depth-discounted lookahead into the inter-core scorer and adding one term per departure: a staging cost for availability, a *core-capacity penalty* for capacity (Section III-D). Capacity is the largest ablation effect: removing both capacity mechanisms costs +87.8% geometric-mean EPR and aborts two circuits (Section IV-D).
- **A capacity condition with a termination-with-success guarantee.** Five preconditions, checked before routing starts so that a violation is reported rather than surfacing as a mid-route abort, make dSABRE provably route every gate rather than merely terminate (Section III-F). The same argument prices the recovery they license— $O(KD_K)$  EPR pairs per remote gate, dSABRE’s first non-vacuous worst-case bound—at a measured cost of 0.2% EPR pairs in geometric mean.
- **Comprehensive empirical validation.** Across Munich Quantum Toolkit (MQT) Bench suites at 25, 36, and 64 qubits, dSABRE reduces geometric-mean EPR consumption by 50.8%, 62.0%, and 49.1% over TelesABRE [12], the one baseline sharing its cost model, architecture and SWAP accounting. The margin survives heavy-hex cores on a ring and on a star, a shared initial allocation, and 360 logical qubits on twenty cores, at near-linear compile time ( $N^{1.05}$ ) (Sections IV-A–IV-C).
- **A controlled comparison across EPR cost models.** `pytket-dqc` [4] distributes by hypergraph partitioning and gate teleportation, so its counts share our unit but not our work per unit. Rebuilding its network to this device’s own links and slots moves the three suite margins to −44.4%, −0.9% and −1.7%, 13 of the 21 corrected distributions exceed the device’s capacity, and under a three-gate entanglement lifetime the narrow 64-qubit margin widens to a 46.5% dSABRE win (Section IV-E).

Three minor contributions support that score. A teleport is

scored only when no front-layer gate has both operands in one core, so local work is drained before an EPR pair is spent and the two scorers—one per-core, one chip-wide—never have to be made commensurable (Section III-A); TelesABRE, which pools both in a single argmin, fires 59.5% of its 64-qubit communication operations while a same-core front gate is still waiting—moments at which dSABRE spends a local SWAP instead (Section V). The inter-core extended set is built BFS layer by layer over the remaining DAG, so that a gate’s depth in the circuit is what discounts its contribution to the lookahead; replacing it with a topological-order set costs 12.4% geometric-mean EPR at 64 qubits (Sections III-E, IV-D). Finally, the initial layout supplies the free slots both capacity mechanisms depend on: stock Qiskit `SabreLayout`, adapted to reserve them in *every* core by a rule reading only vertex degree and remoteness, so it transfers unchanged to non-grid cores (Section III-G).

**Supplementary material.** Evidence that does not fit the page budget accompanies the paper at the URL above: the routing loop in pseudocode and the full proof of the completion theorem; the derivation and measured validation of the complexity bound; per-circuit counts, per-seed counts and circuit provenance for every suite and architecture reported here; the parameter and cost-model sweeps, the 25-qubit ablation and a controlled interaction-degree sweep; and full audits of both baselines—action-level for TelesABRE, code-level for DMapS, with the `pytket-dqc` network correction and entanglement-lifetime sweep.

## II. BACKGROUND

### A. Quantum Circuits

In this paper, a quantum circuit acts on  $n$  qubits through a sequence of *gates*: *one-qubit* (1Q) gates (Hadamard, rotations, ...) and *two-qubit* (2Q) gates, of which the CX (CNOT) gate is the standard primitive. The two classes are together universal—CX and the single-qubit unitaries decompose every  $n$ -qubit unitary exactly [14]—and a finite basis such as  $\{CX, H, T\}$  suffices approximately; it is what the benchmark circuits of Section IV-A are transpiled into. The SWAP gate exchanges the states of two adjacent qubits; it decomposes into three CX gates and is what routing algorithms insert to migrate logical qubit states across the hardware.

A circuit is represented as a *directed acyclic graph* (DAG) in which each node is a gate and each directed edge records a data dependency (qubit produced by gate  $g_1$  consumed by gate  $g_2$ ). For every qubit  $q$ , each gate acting on  $q$  is connected to the *next* gate acting on  $q$ : the second cannot begin until the first has written its output on  $q$ . Figure 1 shows a small example, where *circuit depth* is the length of the longest path through the DAG.

### B. Quantum Teleportation

*Quantum teleportation* [5] transfers the state of a qubit from a sender to a receiver—in the DQC setting, from one core to another over an inter-core link—using a pre-shared EPR pair (a maximally entangled two-qubit state) and two classical bits. The source qubit’s state is destroyed at the sender and

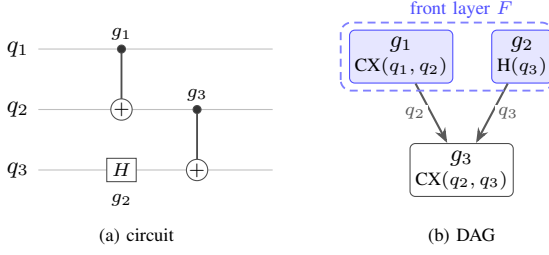


Fig. 1. DAG for a three-gate circuit on qubits  $q_1, q_2, q_3$ .  $CX(q_1, q_2)$  and  $H(q_3)$  share no qubit and have no incoming edges, so both appear in the initial front layer  $F = \{g_1, g_2\}$  and may execute in parallel.  $CX(q_2, q_3)$  depends on both via qubits  $q_2$  and  $q_3$ ; it enters  $F$  only after  $g_1$  and  $g_2$  have been executed. Circuit depth is 2 (two sequential layers).

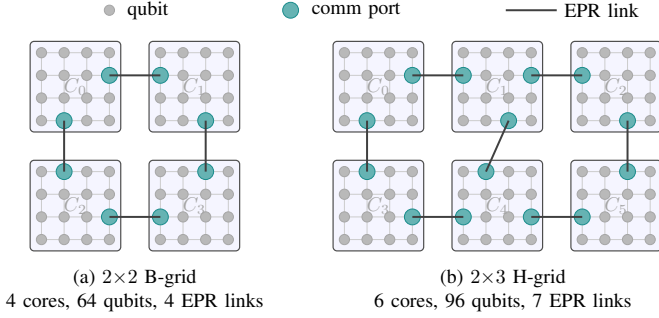


Fig. 2. DQC architectures used in this paper (B-grid and H-grid families introduced by TelesABRE [12]). Small grey circles are qubits; teal circles are inter-core communication ports; bold lines are EPR-channel links between cores. Each core is a  $4 \times 4$  grid. (a)  $2 \times 2$  B-grid. (b)  $2 \times 3$  H-grid.

reconstructed at the receiver; no physical qubit travels across the link. Each EPR pair is consumed upon use, so the *EPR count* is the primary inter-core routing cost in distributed quantum compilation. Throughout this paper, “EPR” as a unit (e.g. “10 EPRs”) denotes EPR-pair count; “EPR pair” is used when emphasising the physical entangled state itself.

Two inter-core teleportation variants arise in DQC compilation. *Teledata* moves a logical qubit to a new core (one EPR pair consumed), after which the qubit participates in local gates there. *Telegate* [6] executes a non-local 2Q gate directly across two cores via a shared EPR pair, without physically relocating either qubit. Which is cheaper is circuit-dependent—teledata amortises over a qubit’s residence in a core, telegate over a group of non-local gates on a shared control bundled onto one entangled state—but the two make different demands on the hardware. A teledata pair is consumed by the teleport that creates it, whereas an amortised telegate needs its link to stay entangled, and to occupy a link qubit, for the span of the gate group it serves. Gate-teleportation counts are therefore contingent on entanglement lifetime and on how many pairs a core can hold at once; Section IV-E measures what those two assumptions are worth.

### C. Distributed Quantum Architecture

We model a DQC architecture  $\mathcal{A}$  abstractly as a weighted graph  $G_{\mathcal{A}} = (V, E, w)$  whose vertices are physical qubits. The vertex set is partitioned into  $K$  cores  $V = V_1 \sqcup \dots \sqcup V_K$ ;

intra-core edges  $E_{\text{intra}}$  describe in-core connectivity and carry weight 1, and inter-core edges  $E_{\text{inter}}$  connect designated *communication ports* on neighbouring cores and carry a much larger weight  $w_{\text{link}} \gg 1$  reflecting the cost of an EPR pair. A communication port is a physical qubit like any other and may also hold a logical (data) qubit between teleportations; the “communication” label only indicates that the port is one end of an inter-core EPR link, not that the slot is reserved. dSABRE treats  $G_{\mathcal{A}}$  as a black box and only queries it through three distance tables: physical  $d_{\text{phys}}$ , intra-core  $d_{\text{intra}}$ , and core-graph  $d_{\text{core}}$ . The router therefore extends to *any* DQC topology — arbitrary intra-core graphs, mixed core sizes, sparse inter-core meshes, hierarchical trees of clusters, etc. — by simply supplying these tables.

For experimental simplicity our reference implementation instantiates  $G_{\mathcal{A}}$  as a *grid-of-grids* [12]: each core is an  $m \times m$  2D nearest-neighbour grid, and the cores themselves form an  $r \times s$  super-grid joined by a small set of inter-core links between specific boundary qubits. Figure 2 shows two concrete topologies used in our experiments.

1) *Hierarchical distance precomputation*: Because intra-core and inter-core costs decouple,  $d_{\text{phys}}$  need never be materialised as a dense all-pairs table: it composes from the two cheap ones. Every  $p \rightarrow q$  path leaves  $p$ ’s core through some communication port and enters  $q$ ’s through another, so with  $d_{\text{port}}$  a port-to-port table over the  $O(K)$  ports, and  $\Pi_i$  the ports of core  $i$ ,  $d_{\text{phys}}(p, q)$  is calculated exactly as

$$\min_{\pi \in \Pi_i, \pi' \in \Pi_j} d_{\text{intra}}(p, \pi) + d_{\text{port}}(\pi, \pi') + d_{\text{intra}}(\pi', q). \quad (1)$$

It cuts precomputation from  $O((KM)^2 \log(KM))$  to  $O(KM^2 + K^2 \log K)$  and resident space from  $\Theta(P^2)$  to  $O(KM^2 + K^2)$ , and queries are memoised, so routing sees  $O(1)$  lookups after the first. This is the only architecture-dependent setup dSABRE performs.

### D. Qubit Routing and SABRE

Let  $C$  be a circuit DAG on  $n$  logical qubits  $q_1, \dots, q_n$ , to be run on an architecture with  $P = |V| \geq n$  physical qubits. An *initial layout* is an injection  $\ell : q_i \mapsto p_{\sigma(i)}$  placing them on distinct physical slots, where  $\sigma$  is a permutation on  $[P] = \{0, 1, \dots, P-1\}$ . A 2Q gate can execute only when its two operands occupy adjacent vertices of the *coupling graph* ( $G_{\mathcal{A}}$  in the distributed case), so *routing* inserts SWAP and teleportation operations—as few as possible—to make every gate executable in turn. Cost model: each local SWAP costs  $c_{\text{swap}}$ ; each inter-core teleport costs  $c_{\text{tele}}$  (one EPR pair).

SABRE [11] is the state of the art for this problem on a single chip: Qiskit [15] selects it by default for both layout and routing, and successive later work has refined its scoring form [16], on single chips and in the multi-core setting [12], [13] alike. Building the distributed router on it is a deliberate choice. A DQC compiler must handle circuits of the size the machine exists to run, on hardware whose link counts, port capacities and core sizes are all finite and all binding; a formulation that is globally optimal on an idealised machine buys its elegance by assuming those constraints away, and pays for it twice—in compile time, and in solutions the device cannot

execute (Section IV-E). Extending a heuristic that production toolchains already run keeps both costs bounded and confines the open question to a single one: what its objective must become when one chip becomes several. LightSABRE [16], the refinement Qiskit ships, selects the SWAP minimising:

$$H = \frac{1}{|F|} \sum_{g \in F} \Delta_F(g) + w \frac{1}{|E|} \sum_{g \in E} \Delta_E(g), \quad (2)$$

where  $F$  is the front layer,  $E$  the extended lookahead set, and  $\Delta(g) = d_{\text{new}}^g - d_{\text{old}}^g$  the change in physical distance between the two qubits of gate  $g$  caused by the candidate SWAP (negative when the SWAP moves them closer);  $d^g$  is shortest-path distance on the coupling graph. All extended-set gates contribute with equal weight; the  $\frac{1}{|E|}$  factor normalises the lookahead term to the same scale as the front term. SABRE also couples its router with an initial-layout optimiser: alternating forward and reverse-circuit passes, each seeded with the previous pass's final layout, anneal towards a layout self-consistent with the circuit's gate structure.

### III. DSABRE

dSABRE is a SABRE-style routing algorithm for multi-core processors. Of the two communication primitives of Section II-B it uses teledata alone, for three reasons. What a telegate does well—settling a group of non-local gates for one EPR pair—teledata also does, since one relocation makes *every* later gate on that qubit local. Realising telegate's advantage therefore means deciding which gates to group, as pytket-dqc does globally by hypergraph partitioning (Section IV-E), and taking that decision *online*—as routing proceeds, from the front layer and a bounded lookahead rather than from the whole circuit—would complicate the score; a single ungrouped telegate, by contrast, fits Eq. 6 directly—no qubit changes core, so  $c_{\text{cap}}$  drops—but then saves a pair only where teledata's amortisation would not have, so we expect little from it. And grouping is what makes a compiler's counts contingent on the device: the link must stay entangled, and hold a link qubit, for the span of the group (Section II-B), which Section IV-E measures.

The core invariants and notation are: (i) the layout  $\ell : q_i \mapsto p_{\sigma(i)}$  of Section II-D assigns logical to physical qubits and the inverse  $\ell^{-1}$  records which logical qubit (if any) currently occupies each physical slot; (ii) the working DAG  $W$  holds the remaining gates (initially the full circuit) and shrinks as gates are executed; (iii) the front layer  $F$  is the set of gates in  $W$  with no unexecuted predecessors, and the extended set  $E$  is a fixed-size sliding lookahead beyond  $F$ . Table I collects every symbol used in this section.

The rest of the section is organised as follows. Sections III-A and III-B give the design as a whole—the two actions the router chooses between, the quantities its score trades off, and the loop that applies one action per iteration. Sections III-C–III-E then fill that score in: the intra-core SWAP heuristic, the teleport candidates and what they cost, and the lookahead set both scorers read. Together they decide every move on a circuit the greedy search can route. Section III-F treats the case it cannot—saturated cores leaving no cheap legal move—where capacity becomes a hard constraint rather

than a preference and a provably terminating transaction retires the blocked gate. Sections III-G and III-H give the initial layout the loop starts from and the cost of the whole procedure.

#### A. Scoring: Potential, Cost, and Capacity

dSABRE has two actions, an intra-core SWAP and an inter-core teleport, and a priority rule rather than a common score decides between them: while any front-layer gate has both operands in one core the router applies an intra-core SWAP (Section III-C); only when none remains does it score teleport candidates (Section III-D). The two scorers are therefore never compared against each other—one is per-core, the other chip-wide.

The intra-core scorer is SABRE restricted to a core, with the depth discount of Eq. 4. The teleport scorer needs more care, because a teleport differs from a SWAP in two ways and each difference adds one term. (i) *Availability*: a SWAP is legal on every edge of the coupling graph, whereas a teleport fires only at a communication port, so a candidate must also carry the intra-core SWAPs that bring its qubit there and any eviction needed to free the port. We score that whole sequence as one *macro-action*, so that a port already occupied makes a candidate dearer rather than deleting it (Section III-D). (ii) *Capacity*: a SWAP exchanges the contents of two slots and leaves the occupied set unchanged, so capacity never binds on a monolithic chip; a teleport moves a qubit into a different core and consumes one of its free slots, so a candidate must be charged for the capacity it takes.

Those two terms sit on an objective inherited from SABRE. Let  $d(g; \ell)$  be the weighted distance between the operands of a 2Q gate  $g$  on  $G_A$  under layout  $\ell$ , minimal exactly when  $g$  is executable, and let the *routing potential*  $\Phi(\ell)$  be  $d$  summed over the front layer plus a lookahead sum over  $E$  in which each gate is discounted.  $\Phi$  is a surrogate, not the true cost-to-go and only its difference is ever evaluated. Writing  $\Delta\Phi(a)$  for the change a candidate  $a$  induces—*negative* when the move leaves the waiting gates closer to executable— $\text{cost}(a)$  for the resource it consumes and  $\text{pen}(a)$  for the capacity constraint  $\Phi$  omits, dSABRE greedily minimises

$$s(a) = \underbrace{\text{cost}(a)}_{\text{resource spent}} + \underbrace{\text{pen}(a)}_{\text{capacity}} + \underbrace{\Delta\Phi(a)}_{\text{progress bought}}. \quad (3)$$

Every candidate in an iteration spends exactly one EPR pair, so that cost is constant across the arg min and does not appear.

The capacity term is not a tie-breaker added to a distance heuristic: it is *graded* in the landing core's free-slot count rather than switching on at a fixed occupancy. A graded penalty can express preference but never guarantee feasibility, so we also put (cf. Section III-F) two hard constraints behind it. A *legality floor* admits a teleport only if its destination core holds at least  $\phi_1=2$  free slots when it is scored; and a *feasibility* condition on the instance, checked once before routing, is what makes the router provably complete. The score term is then left to choose among already-legal moves.

#### B. Workflow

Figure 3 sketches the main routing loop. Each iteration runs four phases: (P1) drain  $F$  (execute 1Q gates and any 2Q



TABLE I  
NOTATION FOR SECTION III, WITH THE DEFAULT VALUE OF EACH HYPERPARAMETER. EVERY SCORING PARAMETER IS USED UNCHANGED IN EVERY EXPERIMENT AND ON EVERY ARCHITECTURE;  $L_{\text{deadlock}}$  IS READ OFF THE ARCHITECTURE RATHER THAN TUNED PER SUITE, AND THE CAPACITY CONSTANTS ARE STRUCTURAL,  $r \geq 2$  BEING A HYPOTHESIS OF THE THEOREM OF SECTION III-F.

| Symbol                              | Meaning                                              | Default       |
|-------------------------------------|------------------------------------------------------|---------------|
| <b>Routing state</b>                |                                                      |               |
| $\ell, \ell^{-1}$                   | layout $q_i \mapsto p_{\sigma(i)}$ , and its inverse | —             |
| $W$                                 | working DAG of unexecuted gates                      | —             |
| $F, E$                              | front layer; extended (lookahead) set                | —             |
| $F_c, E_c$                          | their restrictions to core $c$                       | —             |
| $K, M$                              | number of cores; qubits per core                     | —             |
| $n, P$                              | logical qubits in $C$ ; physical qubits $ V =KM$     | —             |
| <b>Distances and scores</b>         |                                                      |               |
| $G_A$                               | full-chip graph (Section II-C)                       | —             |
| $d(g; \ell)$                        | weighted distance between operands of $g$            | —             |
| $d_{\text{intra}}, d_{\text{core}}$ | intra-core and core-graph distance                   | —             |
| $D_M, D_K$                          | intra-core and core-graph diameter                   | —             |
| $\text{dep}(g)$                     | depth of $g$ (BFS layer, inter-core set)             | —             |
| $\Phi$                              | routing potential (Section III-A)                    | —             |
| $\Delta_F$                          | change in $d$ over the front gate                    | —             |
| $\tilde{\Delta}_E$                  | decay-weighted change over $E$ , Eq. 4               | —             |
| $d_{\text{prep}}$                   | staging + eviction SWAPs before a teleport           | —             |
| $c_{\text{cap}}$                    | capacity penalty on the landing core                 | —             |
| $f_{\text{next}}$                   | free slots in the landing core $c_{\text{next}}$     | —             |
| $\pi_s, \pi_d$                      | source/destination communication ports               | —             |
| $n_s$                               | staging slot: neighbour of $\pi_s$ nearest $p_1$     | —             |
| <b>Hyperparameters</b>              |                                                      |               |
| $c_{\text{swap}}$                   | SWAP cost                                            | 3             |
| $c_{\text{tele}}$                   | teleport (EPR) cost                                  | 10            |
| $\tau$                              | capacity threshold                                   | 3             |
| $c_{\text{pen}}$                    | capacity penalty multiplier                          | 15            |
| $w_{\text{link}}$                   | inter-core link weight                               | 10            |
| $w_e$                               | extended-set weight                                  | 0.25          |
| $L$                                 | extended-set capacity                                | 20            |
| $\gamma$                            | lookahead decay                                      | 0.9           |
| $L_{\text{deadlock}}$               | stalled iterations before rollback                   | $4D_K(D_M+1)$ |
| <b>Capacity constants</b>           |                                                      |               |
| $r$                                 | capacity reserve, Eq. 8                              | 2             |
| $\phi_1$                            | legality floor: free slots the destination must hold | 2             |

gate whose operands are already adjacent in the same core, iterated to a fixed point); **(P2)** partition the remaining front into  $F_{\text{intra}}$  (both operands in one core) and  $F_{\text{inter}}$  (operands in different cores); **(P3)** if  $F_{\text{intra}} \neq \emptyset$  apply one intra-core SWAP (Section III-C), else score teleportation candidates over  $F_{\text{inter}}$  and apply the cheapest (Section III-D); **(P4)** checkpoint  $(\ell, W)$  if  $|W|$  decreased, otherwise, after  $L_{\text{deadlock}}$  stalled iterations, restore the checkpoint and retire the stuck gate by the guaranteed transaction of Section III-F. Two other states hand P3 straight to the same transaction without waiting for  $L_{\text{deadlock}}$ : an empty candidate list, meaning no neighbouring core clears the legality floor, and a candidate set every member of which is rejected at execution time (Section III-D). Neither is a stall a local SWAP could resolve, so there is nothing to gain by stalling first.

### C. Intra-Core SWAP Heuristic

When the front contains a gate whose operands share a core but are not adjacent, dSABRE applies the standard SABRE heuristic restricted to that core. Let  $F_c = \{g \in F_{\text{intra}} : \text{core}(g) = c\}$  be the front gates in core  $c$  and  $E_c = (g_0, g_1, \dots)$  the corresponding intra-core extended set.  $E_c$  is the extended set  $E$  restricted to core  $c$ —the gates of  $E$  with both operands

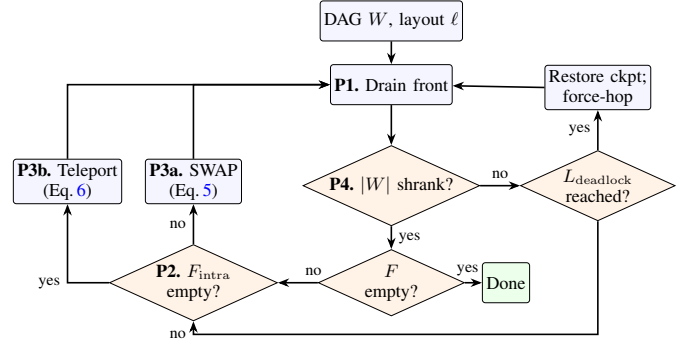


Fig. 3. The dSABRE routing workflow. Each iteration drains the front layer—executing 1Q gates and adjacent intra-core 2Q gates (P1)—classifies the remaining front into intra-core and inter-core gates (P2), and applies either the best intra-core SWAP (P3a,  $\arg \min H_{\text{intra}}$ ) or the best gate-driven teleport candidate (P3b,  $\arg \min s$ ). It checkpoints when  $|W|$  falls (P4); after  $L_{\text{deadlock}}$  stalled iterations it instead restores the checkpoint and retires the stuck gate by the guaranteed transaction of Section III-F. The same transaction is also reached directly from P3b when no candidate is legal or none commits.

resident there, carrying  $E$ 's own depths—so one construction (Section III-E) feeds both scorers and  $\text{dep}(g)$  means the same thing in each. The submitted design instead built a separate per-core set by its own traversal; the online appendices compare the two. With  $\text{dep}(g_i)$  the BFS layer  $g_i$  occupies in  $E$ , the lookahead contribution is

$$\tilde{\Delta}_E^c = \sum_{i=0}^{|E_c|-1} \gamma^{\text{dep}(g_i)} (d_{\text{new}}^{g_i} - d_{\text{old}}^{g_i}), \quad (4)$$

where  $\gamma \in (0, 1]$  is the lookahead decay and the distance terms use the intra-core graph; the tilde marks the departure from SABRE's flat extended-set weighting (Section III-A). For every neighbour pair  $(u, v)$  within a core, the score

$$H_{\text{intra}} = \frac{\Delta_F^c}{|F_c|} + w_e \frac{\tilde{\Delta}_E^c}{|E_c|} \quad (5)$$

is SABRE's objective with that weighting; the lowest-scoring SWAP is applied, ties broken by enumeration order. Because the scorer sees one core and  $O(M)$  candidate pairs, per-core selection is trivially parallel across active cores.

### D. Inter-Core Teleportation Scoring

This is where the *capacity-aware macro-action* of Section III-A is instantiated: a candidate is the whole sequence—staging SWAPs, evictions, teleport—and it is priced by what that sequence costs plus what it takes from the core it lands in.

**Operational definition.** Executing a candidate is deterministic given the state at the moment it is scored. The destination port  $\pi_d$  need not be empty: an occupant is evicted to the nearest free slot in  $c_{\text{next}}$  by the same rule applied to the source port  $\pi_s$ . The legality floor of Section III-A leaves a candidate destination two free slots when it is scored, so that eviction cannot fail. A fully-occupied *source* core, by contrast, cannot teleport at all—clearing  $\pi_s$  needs a slot to displace its occupant into—so one qubit is first teleported out of it, into whichever neighbouring core has room. That hop can

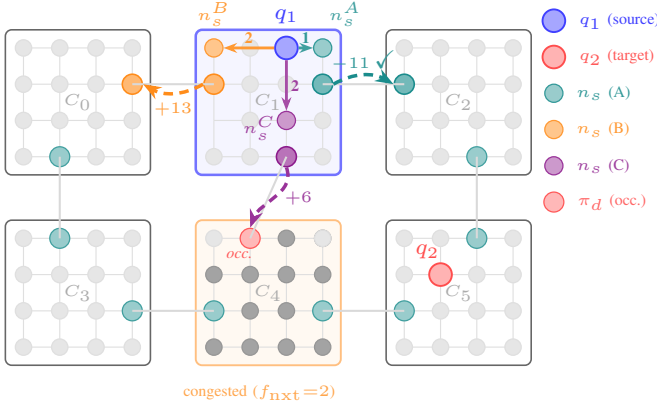


Fig. 4. How dsABRE chooses where to teleport a qubit. A single cross-core gate  $g=(q_1, q_2)$  is pending, with  $q_1$  in core  $C_1$  (blue border) and its partner  $q_2$  in  $C_5$ ; the router scores the three links leaving  $C_1$  and takes the cheapest by Eq. 6. Executing any one of them takes two steps: intra-core SWAPs first move  $q_1$  to the staging slot  $n_s$  (count on the solid arrow), then a teleport along the link  $(\pi_s, \pi_d)$  carries it into the next core (dashed arc, annotated with the resulting score  $s$ ). The three candidates isolate one decision factor each, and correspond row by row to Table II. **A** (teal) moves towards  $C_5$  and wins,  $s_A = -11$ . **B** (orange) moves to  $C_0$ , away from  $C_5$ , so its front-layer term turns positive. **C** (violet) makes the same progress as A but lands in the nearly-full core  $C_4$  (orange border, two free slots), where the capacity penalty outweighs the progress. The red port marked *occ.* is already occupied, so C also pays an extra eviction SWAP.

TABLE II

SCORE BREAKDOWN FOR THE THREE CANDIDATES OF THE RUNNING EXAMPLE (FIGURE 4). LOWER IS BETTER; CANDIDATE A IS SELECTED.

| Candidate | next core | direction       | $d_{\text{prep}}$ | $c_{\text{cap}}$ | $\Delta_F$ | score $s$ |
|-----------|-----------|-----------------|-------------------|------------------|------------|-----------|
| A         | $C_2$     | towards $C_5$   | 1                 | 0                | -12        | -11       |
| B         | $C_0$     | away from $C_5$ | 2                 | 0                | +11        | 13        |
| C         | $C_4$     | towards $C_5$   | 3                 | 15               | -12        | 6         |

land in  $c_{\text{next}}$  and spend one of its two slots, so both cores are re-checked afterwards: a candidate whose preconditions no longer hold when it executes is rolled back at zero cost, and the next-cheapest is tried in its place—a safety net the conditions of Section III-F render provably unnecessary, as the online appendices show.

For each gate  $g \in F_{\text{inter}}$  on logical qubits  $q_1, q_2$ , let  $c_{\text{src}}$  and  $c_{\text{tgt}}$  be the cores currently hosting  $q_1$  and  $q_2$  respectively (i.e.  $\ell(q_1) \in c_{\text{src}}$  and  $\ell(q_2) \in c_{\text{tgt}}$ , where  $\ell$  is the current layout). Both endpoints are tried as teleportation sources. For the qubit chosen as source (say  $q_1$  at physical position  $p_1 = \ell(q_1)$ ), the router enumerates every neighbouring core  $c_{\text{next}}$  and every inter-core link  $(\pi_s, \pi_d)$  between  $c_{\text{src}}$  and  $c_{\text{next}}$ . Each such triple  $(q_1, c_{\text{next}}, (\pi_s, \pi_d))$  is one *candidate*; Figure 4 illustrates the structure. Executing a candidate involves two steps, **plus** **eviction of either communication port if occupied** (operational definition above): (1) intra-core SWAPs that move  $q_1$  from  $p_1$  to the *staging slot*  $n_s$ , the neighbour of  $\pi_s$  closest to  $p_1$ ; and (2) a teleportation that consumes the EPR pair on link  $(\pi_s, \pi_d)$  and delivers  $q_1$  from  $n_s$  to  $\pi_d$  in  $c_{\text{next}}$ . **Both steps belong to the action and are scored together**, so a teleport is available from *anywhere* in the source core and an occupied port raises a candidate’s score rather than removing it (Section V contrasts this with TelesABRE). Each candidate is scored as:

$$s = \underbrace{d_{\text{prep}}}_{\text{staging}} + \underbrace{c_{\text{cap}}}_{\text{capacity}} + \underbrace{\Delta_F + w_e \tilde{\Delta}_E}_{\Delta\Phi \text{ (front + lookahead)}}, \quad (6)$$

where lower scores are preferred. The last group is SABRE’s front-plus-lookahead objective (Eq. 2), the whole score on a monolithic chip; the first two are the availability and capacity terms of Section III-A. Unlike Eq. 5 this score carries no denominators:  $\Delta_F$  is taken over the single front gate the candidate is scored for, and  $\tilde{\Delta}_E$  is already scaled by  $\gamma^{\text{dep}(g)}$  and capped at  $|E|_{\text{max}}$ , so there is no cardinality to divide out. Each term is defined as follows.

**Staging cost**  $d_{\text{prep}}$  charges the intra-core SWAPs the move needs: the walk taking  $q_1$  from  $p_1$  to staging slot  $n_s$ , plus eviction SWAPs to free the communication ports if occupied:

$$d_{\text{prep}} = d_{\text{intra}}(p_1, n_s) + d_{\text{evict}}(\pi_s) + d_{\text{evict}}(\pi_d).$$

**Capacity penalty**  $c_{\text{cap}} = c_{\text{pen}} \max(0, \tau - f_{\text{nxt}})$  discourages routing into nearly-full cores.  $f_{\text{nxt}}$  is the number of free slots in the immediate landing core  $c_{\text{next}}$  (not the final partner-qubit core  $c_{\text{tgt}}$ ),  $\tau$  the capacity threshold, and  $c_{\text{pen}}$  a cost multiplier. This discourages progressive crowding of high-demand cores and empirically reduces the cascading evictions associated with saturated landing cores. **The two capacity mechanisms act at different points: the legality floor is a filter**, so a core with fewer than  $\phi_1$  free slots never becomes a candidate, while  $c_{\text{cap}}$  grades the survivors. At the defaults the surviving band is narrow—a legal landing core has  $f_{\text{nxt}} \geq 2$ , so the penalty is  $c_{\text{pen}}$  exactly when  $f_{\text{nxt}}=2$  and zero above—which is why it costs little once the floor is in place (Section IV-D).

**Front-layer change**  $\Delta_F = d_{\text{new}}^g - d_{\text{old}}^g$  is the change in weighted physical distance between  $q_1$  and  $q_2$  after the teleport (negative when they are brought closer), computed on the full-chip graph  $G_A$  (intra-core edges weight 1, inter-core links weight  $w_{\text{link}}$ ). Exactly one front-layer gate involves  $q_1$ : gates sharing a qubit are DAG-ordered, so at most one can be unblocked at a time, and  $g$  is already that gate by the premise of this scoring step.

**Decay-weighted lookahead.**  $\tilde{\Delta}_E$  is Eq. 4 over the whole extended set  $E$  (Section III-E) rather than a single core’s restriction  $E_c$ , with one adaptation for the inter-core scorer: the sum is restricted to gates involving the teleported qubit, the only ones the move affects.

1) *Running example:* Figure 4 and Table II score the three outgoing-link candidates on the  $2 \times 3$  H-grid at defaults, isolating one factor each, **and show the three groups of Eq. 3 deciding between them:** A wins on the cheapest staging plus the largest progress; B moves away from  $C_5$ , turning  $\Delta_F$  against it; C buys A’s progress but lands in a nearly-full core, where the capacity term alone reverses the decision.

### E. Inter-Core Extended-Set Construction

The lookahead in Eq. 6 is the same  $\tilde{\Delta}_E$  as Eq. 4, with  $\text{dep}(g)$  now read off the inter-core set  $E$  rather than an intra-core wire; how  $E$  is filled is therefore what decides whether the discount means anything. SABRE [11] fills  $E$  in topological

order, interleaving all wires uniformly; TeleSABRE [12] uses BFS layers but within each layer emits gates in arbitrary order and weights the  $g$ -th by  $(1 + g/10)$ , growing with emission index rather than decaying with depth. Both can deprioritise the very follow-on gates a candidate teleport would help.

dSABRE expands the remaining DAG by BFS layer, assigning every gate of layer  $k$   $\text{dep}(g)=k$  and emitting, within a layer, the gates sharing a qubit with the front first. It adds no hyperparameter and runs in  $O(|E|_{\max})$  time per call when remaining in-degrees are maintained incrementally.

The construction matters structurally: a gate’s influence is  $\gamma^{\text{dep}(g)}$ , so the lookahead is meaningful only if the gates surviving the size- $L$  truncation are the shallow ones. BFS layering guarantees that—layer  $k+1$  is emitted only once layer  $k$  is exhausted—whereas a topological sweep can spend the whole budget on one deep wire and omit a depth-1 gate on another, which is why its cost grows with circuit size (Section IV-D).

### F. Capacity Conditions and a Guaranteed Transaction

One state defeats the score of Eq. 3: every cheap action blocked by saturated cores, or no action reducing  $|W|$  at all. dSABRE meets it with the checkpoint–rollback plumbing SABRE-style routers carry, and a transaction that provably retires the stuck gate. Every iteration that reduces  $|W|$  checkpoints  $(\ell, W)$ ; after  $L_{\text{deadlock}}$  stalled iterations—or immediately, if no candidate is legal—the router restores the checkpoint and hands the gate to that transaction.

**Three capacity conditions.** Write  $F = P - n$  for the free slots the architecture leaves the circuit. The first condition is on the *instance*,

$$F \geq rK + 1, \quad (7)$$

with reserve  $r=2$  by default. Every SWAP and teleport conserves  $F$  exactly, so this is a property of architecture and circuit alone: checked once before routing, and true at every step thereafter. Its  $+1$  guarantees a *donor*—if every core held at most  $r$  free slots the total would be at most  $rK$ , so some core always holds  $r+1$  and can give one away without dropping below  $r$  itself.

The second is the per-core *reserve*

$$\text{free}(c) \geq r \quad \text{for every core } c, \quad (8)$$

the state from which the transaction below can plan. It is *not* maintained: ordinary scored moves may spend a core below it. It holds at exactly two kinds of moment—once at entry, where the initial layout is brought up to it, and on each entry to a transaction, which re-establishes it for itself by relaying slack from a donor and exits having restored it. Eq. 7 is what makes that relay always possible, so the reserve is paid for when it is used rather than on every iteration.

The third is what ordinary scoring must respect, and it is the weakest: a teleport is admitted only if its destination holds  $\phi_1=2$  free slots when it is scored, so one survives the arrival. Zero free slots is what actually breaks routing: a qubit in a full core generally cannot leave it, since clearing the outgoing port needs somewhere to displace its occupant. Forbidding that costs one comparison per candidate, and together with the two

conditions above yields an invariant—no core is ever full—that makes every scored move executable as scored.

Given Eq. 8, any remote gate can be resolved by a *transaction*: a plan executed all-or-nothing, which either retires its gate and leaves Eq. 8 restored, or leaves the layout exactly as it found it. The plan is to choose a meeting core minimising hop count plus the cost of relaying slack to it, walk the operands there, and execute. Every hop stays legal—a shortest core path touches each intermediate core twice, and an arrival leaves it at  $r-1 \geq 1$ , still enough to clear the destination port, with the departure restoring it—and the transaction exits having restored Eq. 8, or rolls back at zero cost. Hop count and relay cost are its whole objective, not Eq. 3: its job is feasibility, not EPR economy.

**Theorem (termination with success).** *If  $r \geq 2$ , the core graph and every core’s coupling graph are connected, every core holds at least  $r+3$  physical qubits, Eq. 7 holds, and checkpoint–rollback is enabled, then dSABRE routes every gate of the input circuit: it terminates, and it does not abort.*

*Proof (Sketch):* We use induction on the unrouted-gate count. From any reachable state Eq. 7 supplies a donor, so relaying its slack re-establishes Eq. 8, from which the whole plan is legal. The transaction therefore retires its target gate, and the count strictly decreases, so at most one transaction fires per gate; between two of them the loop runs at most  $L_{\text{deadlock}}$  non-progressing iterations before the next one does. Charging each gate its stall window, its own transaction and one progressing iteration bounds the whole loop at  $L_{\text{deadlock}}+2$  iterations per gate—the two being the iteration in which its transaction fires and the one in which some gate retires, the stall window covering the rest. This bounds *iterations*, which is what Eq. 9 costs: an ordinary iteration applies one intra-core SWAP per active core or one teleport, so it spends at most one EPR pair, while the transaction’s iteration spends the  $O(KD_K)$  priced below. The induction runs on Eq. 7 rather than Eq. 8, which is why the reserve need not be maintained. It also prices the transaction. Co-locating the operands costs  $D_K$  hops, and clearing room at the meeting core at most  $2D_K$  more. Re-establishing Eq. 8 first is the dominant term: ordinary moves can leave every core one slot short, and each unit of slack travels up to  $D_K$  hops from its donor, so the repair costs  $O(KD_K)$  in the worst case—on a core path whose slack all sits at one end,  $\Theta(K^2)$ . Even a circuit drained entirely by transactions therefore costs  $O(KD_K)$  EPR per remote gate, the first non-vacuous upper bound dSABRE has had. It is loose: what the transaction actually spends is measured in Section IV-D2.

All five hypotheses are checked at entry, so a violation is reported before routing starts rather than as an abort mid-route, and the initial layout is brought up to Eq. 8 at the same point.

### G. Initial Layout

Both the capacity penalty and the guaranteed transaction depend on the initial layout leaving free slots; this is how it is produced. dSABRE bootstraps from Qiskit SabreLayout [11] run on the full architecture graph  $G_A$  (intra-core edges plus inter-core links), with two adaptations for the distributed setting.

*Per-core corner reservation.* SabreLayout packs each core to capacity, which stalls the inter-core scorer—an occupied port cannot accept a teleport without first evicting its occupant—and trips the capacity penalty on every candidate landing there. We therefore withhold from SabreLayout’s coupling map the  $k$  most-remote *corner* qubits of *every* core: the minimum-degree vertices farthest, by total shortest-path distance, from all others.  $k$  is adaptive, the largest value in  $\{r, \dots, 4\}$  holding the usable-slot fill  $n_q/(N_{\text{cores}}(m^2 - k))$  at or below 0.80— $k=4$  on the 25/36/100/200-qubit suites,  $k=2$  on the denser 64- and 360-qubit ones. The floor at  $r$  is what makes Eq. 8 hold on the produced layout by construction rather than by coincidence, and Eq. 7 is exactly the condition under which it is affordable. Reserving *per core* is what matters: under core-major numbering a whole-chip rule reserves slots in one core and packs every other. Since the rule reads only vertex degree and remoteness it transfers to any core shape, selecting leaf qubits on the heavy-hex cores of Section IV-B where it selects corners on a grid.

*Forward-backward-forward schedule.* We run three passes—forward, then the reversed DAG, then forward again—and keep the better of the two forward passes by EPR. The backward pass pre-positions qubits near their early interaction partners, which the third warm-starts from: SABRE’s backward refinement [11], and better than pure-forward warm-restart. Best of three SabreLayout seeds is reported.

## H. Complexity

Write  $N=|C|$  for the gate count,  $D_M$  and  $D_K$  for the intra-core and core-graph diameters, and  $A, B, T$  for the intra-core SWAPs, the teleports, and the iterations the router performs; the rest is as in Table I. Distance tables are precomputed once per architecture in  $O(KM^2 + K^2 \log K)$  (Section II-C) and are not charged per circuit.

Scoring is charged per *applied move* rather than per iteration, since an iteration applies one SWAP in each active core and charging both per-iteration and per-move would count that parallelism twice. A gate needs  $O(D_M)$  SWAPs and  $O(D_K)$  teleports, so  $A = O(ND_M)$ ,  $B = O(ND_K)$  and  $T = O(A + B)$ . Scoring one intra-core SWAP costs  $O(|F_c| + L)$ : a candidate moves at most *two* front-layer operands, since front gates are qubit-disjoint, leaving every other term of  $\Delta_F$  identically zero. Scoring one teleport costs  $O(Mn + P)$ :  $O(n)$  candidates at  $O(M + L)$  each—the  $M$  is the scan for an eviction target—over an  $O(P)$  free-slot census. Both lookahead sets are cheap once in-degrees are carried between calls:  $O(n + L)$  for the inter-core set, and  $O(L)$  for every  $E_c$  together, one pass over it. Hence

$$T_{\text{total}} = O\left(\underbrace{N(D_M + D_K)(n + L + M)}_{\text{lookahead}} + \underbrace{ND_M(M + L)}_{\text{SWAP scoring}} + \underbrace{ND_K(Mn + P)}_{\text{teleport scoring}}\right) \quad (9)$$

—unconditionally, because the theorem of Section III-F caps the loop at  $L_{\text{deadlock}} + 2$  iterations per gate and so supplies a budget instead of assuming one.  $D_M + D_K$  is  $O(\sqrt{M} + \sqrt{K})$

on a grid-of-grids but linear in  $K$  on a core ring, and the bound is linear in  $N$ —which the router measures close to,  $N^{1.05}$  across every suite here. Restricting intra-core scoring to core-local edges further reduces the front-layer term by up to a factor  $K$  over flat SABRE [11] on the same  $P$  qubits.

## IV. EXPERIMENTAL EVALUATION

### A. Setup

**Benchmarks.** We use three circuit sets from the MQT Bench suite [17], all in the IBM native-gate set (Qiskit opt3 transpilation, measurements and barriers removed).

*Suite 1 (25-qubit):* quantum amplitude estimation (AE), quantum Fourier transform (QFT), quantum neural network (QNN), random circuit (Random), GHZ state preparation, and graph-state preparation.

*Suite 2 (36-qubit):* six circuits spanning a broader range of algorithmic families and CX densities: Bernstein–Vazirani (BV, linear chain), Deutsch–Jozsa (DJ, oracle), W-state preparation (cascaded star), VQE with SU(2) ansatz (layered variational), QAOA (dense all-to-all), and quantum phase estimation (QPEexact, hierarchical).

*Suite 3 (64-qubit):* the same six circuit families as Suite 1 scaled to 64 logical qubits (AE, QFT, QNN, Random, GHZ, Graphstate), run on the larger H-grid architecture, extended with three further circuits that complete the algorithmic-class coverage at the largest scale: QPEexact (phase estimation), QAOA (dense max-cut cost layers with shared-control ZZ structure), and a 16-bit Multiplier (arithmetic, the suite’s largest circuit at 13 040 CX).

**Architecture.** Suites 1 and 2 use a  $2 \times 2$  B-grid: four  $4 \times 4$  cores (64 physical qubits, 4 EPR links). Suite 3 uses a  $2 \times 3$  H-grid: six  $4 \times 4$  cores (96 physical qubits, 7 EPR links). See Figure 2 for the full qubit-level topology. The 25q suite on the B-grid and the 64q suite on the H-grid follow the benchmark setup of TeleSABRE [12]. Section IV-C1 adds two further architectures, built from IBM heavy-hex cores wired as a ring and as a star, to test whether the router’s advantage depends on the grid-of-grids structure above.

**Baselines.** TeleSABRE [12] is the baseline every headline number is measured against. It shares dSABRE’s cost model, physical architecture, and intra-core SWAP accounting. It is also a SABRE-style router that jointly optimises intra-core SWAPs, teledata (qubit moves), and telegate (remote CX execution) operations, run using its compiled C binary with the `optimize_initial` flag enabled (built-in layout heuristic + 3-success sampling) on the same device.

We also compare against `pytket-dqc` [4], a Quantum library that partitions a circuit’s interaction hypergraph with KaHyPar and then *embeds* each hyperedge as a sequence of gate teleportations sharing one EPR pair—a Steiner tree or cover embedding spanning the servers the hyperedge touches—so multiple non-local gates on the same wire amortise onto a single pair. The machine-model caveats listed in Section I, all of them in `pytket-dqc`’s favour, are large enough that the comparison needs a section of its own (Section IV-E). The third baseline, DMapS [13], is the closest peer—a SABRE-derived router sharing our joint EPR-plus-SWAP objective, differing mainly in its cross-core primitive,



a two-EPR remote SWAP. Section V gives the head-to-head: 2–4× fewer EPR pairs at 25q, 1.6–1.8× at 64q.

*What one EPR pair buys.* All three tools report in the same unit, one consumed EPR pair—but what each unit *buys* differs, and no cross-tool comparison is meaningful without stating the difference. dSABRE uses *teledata*: one EPR per inter-core hop relocates a logical qubit, after which subsequent intra-core gates on it are free. TeleSABRE uses that primitive and *telegate*, and we count each operation once. *pytket-dqc*, not a router but a static partitioner, amortises many gates onto one pair under an unbounded-entanglement-lifetime assumption (Section II-B). Its column is therefore unit-consistent with the others but not work-consistent, in the sense given in the Introduction; the online appendices tabulate what a pair buys under each tool’s model.

**Parameters.** Table I lists all hyperparameters; the defaults are used unchanged on every suite and architecture, and the recovery budgets are read off the architecture rather than set, so no suite carries a tuned value. Each router uses its authors’ convention: best-of-3 seeds for dSABRE and TeleSABRE [11], [12], best-of-5 for *pytket-dqc* [4], which reflects operational cost rather than per-attempt variance.

## B. Main Results

TeleSABRE [12] supports three operation types: intra-core SWAPs, *teledata* (qubit movement, 1 EPR), and *telegate* (remote CX execution via the cat-entangler protocol, 1 EPR). We report TeleSABRE’s EPR count as the sum of *teledata* and *telegate* operations, so each EPR pair is counted exactly once. Unless stated otherwise, dSABRE is run with its default initial-layout configuration (Section III-G). Subsequent tables and ablations inherit this default. We report two metrics in the main tables: EPR pairs and intra-core SWAPs. This subsection carries the headline TeleSABRE comparison over the three benchmark suites and the large-QFT points; Section IV-C then repeats it on architectures that are not grids at either level, pushes it to 360 logical qubits, and asks whether it survives a shared initial layout and a harsher cost model.

**25q.** dSABRE reduces geometric-mean EPRs over TeleSABRE by **50.8%** (Table III). Per-circuit reductions range from –11.5% on AE to –84.6% on Graphstate.

**36q.** dSABRE reduces geometric-mean EPRs by **62.0%** (Table III), with per-circuit reductions of –25% to –83% and no circuit at parity. The simple structured circuits BV, VQE-SU2, and W-state are essentially solved (1 EPR on BV; 6–10 EPRs on the others). dSABRE also uses fewer intra-core SWAPs than TeleSABRE on every circuit of this suite, 83 against 173 in geometric mean.

**64q.** The H-grid suite is where the gap from TeleSABRE is most consequential (Table III). The suite comprises the six TeleSABRE-benchmark circuits plus QPEexact, QAOA, Multiplier. dSABRE reduces gmean EPRs by **49.1%** over the eight circuits on which TeleSABRE converges. On Random, TeleSABRE fails to converge on any of three seeds; dSABRE completes it under default scoring alone, without once invoking the guaranteed transaction (Table V).

TABLE III

MAIN RESULTS: DSABRE AGAINST TELESABRE, THE ONE BASELINE THAT SHARES ITS COST MODEL, PHYSICAL ARCHITECTURE, AND INTRA-CORE SWAP ACCOUNTING. EPR = TELEDATA + TELEGATE FOR TELESABRE, TELEDATA FOR DSABRE; SWAP COUNTS INTRA-CORE SWAPS. NEGATIVE  $\Delta$ EPR FAVOURS DSABRE. TELESABRE IS BEST-OF-3 SEEDS, DSABRE BEST-OF-3 SABRELAYOUT SEEDS UNDER THE FWD→BWD→FWD SCHEDULE. \*TELESABRE FAILS TO CONVERGE; THE GEOMETRIC MEAN IS TAKEN OVER THE CIRCUITS IT COMPLETES, AND WHERE THAT IS NOT THE WHOLE SUITE A SECOND ROW GIVES DSABRE’S MEAN OVER EVERY CIRCUIT IT ROUTED.

| Circuit                                                              | CX    | TeleSABRE |      | dSABRE |       | $\Delta$ EPR (%) |
|----------------------------------------------------------------------|-------|-----------|------|--------|-------|------------------|
|                                                                      |       | EPR       | SWAP | EPR    | SWAP  |                  |
| <i>25 logical qubits, B-grid 2×2 of 4×4 cores (64 physical)</i>      |       |           |      |        |       |                  |
| ae                                                                   | 558   | 26        | 292  | 23     | 288   | −11.5            |
| ghz                                                                  | 24    | 4         | 36   | 1      | 4     | −75.0            |
| graphstate                                                           | 25    | 13        | 47   | 2      | 15    | −84.6            |
| qft                                                                  | 580   | 46        | 405  | 33     | 332   | −28.3            |
| qnn                                                                  | 1223  | 54        | 659  | 47     | 661   | −13.0            |
| random                                                               | 1124  | 271       | 1248 | 181    | 1208  | −33.2            |
| <b>gmean (6)</b>                                                     |       | 31.1      | 234  | 15.3   | 129   | <b>−50.8</b>     |
| <i>36 logical qubits, same B-grid</i>                                |       |           |      |        |       |                  |
| bv                                                                   | 17    | 6         | 39   | 1      | 13    | −83.3            |
| dj                                                                   | 35    | 4         | 63   | 3      | 23    | −25.0            |
| qaoa                                                                 | 1200  | 268       | 1165 | 134    | 1017  | −50.0            |
| qpeexact                                                             | 1019  | 190       | 883  | 62     | 612   | −67.4            |
| vqe_su2                                                              | 105   | 28        | 134  | 9      | 45    | −67.9            |
| wstate                                                               | 70    | 13        | 79   | 6      | 40    | −53.8            |
| <b>gmean (6)</b>                                                     |       | 27.6      | 173  | 10.5   | 83    | <b>−62.0</b>     |
| <i>64 logical qubits, H-grid 2×3 of 4×4 cores (96 physical)</i>      |       |           |      |        |       |                  |
| ae                                                                   | 1962  | 353       | 1494 | 165    | 1325  | −53.3            |
| ghz                                                                  | 63    | 16        | 132  | 6      | 39    | −62.5            |
| graphstate                                                           | 64    | 62        | 281  | 15     | 88    | −75.8            |
| qft                                                                  | 1966  | 312       | 1711 | 180    | 1480  | −42.3            |
| qnn                                                                  | 8126  | 889       | 5471 | 539    | 5260  | −39.4            |
| random*                                                              | 1627  | —         | —    | 732    | 3603  | —                |
| qpeexact                                                             | 2139  | 289       | 1706 | 187    | 1576  | −35.3            |
| qaoa                                                                 | 3920  | 956       | 4189 | 555    | 3734  | −41.9            |
| multiplier                                                           | 13040 | 1601      | 9654 | 1285   | 8969  | −19.7            |
| <b>gmean (8)</b>                                                     |       | 284.5     | 1564 | 144.7  | 1081  | <b>−49.1</b>     |
| <b>gmean (9)</b>                                                     |       | —         | —    | 173.3  | 1236  | —                |
| <i>QFT scalability, 5×5 cores throughout, core count 6 → 12 → 20</i> |       |           |      |        |       |                  |
| 100q                                                                 | 3420  | 248       | 2414 | 201    | 2452  | −19.0            |
| 200q                                                                 | 7220  | 1232      | 7089 | 766    | 6482  | −37.8            |
| 360q*                                                                | 13300 | —         | —    | 1890   | 15606 | —                |

## C. Generality and Scale

*1) Architecture independence: heavy-hex cores on a ring and on a star:* Section II-C models the architecture as an abstract weighted graph queried only through three distance tables, but everything above runs on a grid of grids. If the scoring or the capacity mechanism tacitly assumed grid structure at either level, that generality claim would be empty, so we vary both levels at once. The core becomes a 27-qubit IBM heavy-hex tile of the Falcon family [18], which shares none of the grid’s regularity—degrees over {1, 2, 3}, six degree-1 leaves, diameter 12 against the 4×4 grid’s 6—and four such tiles (108 physical qubits) are wired as a *ring* (4 links) and as a *star* (3 links, core 0 a hub every inter-core route must pass). The star is the harsher case, and the one in which a

scorer that quietly assumed a homogeneous core graph would break. Nothing is specialised for either: the corner reservation of Section III-G reads vertex degree and remoteness, so it selects leaf qubits where it selected grid corners, and takes  $k=4$  off the 59% fill on its own.

dSABRE keeps its advantage on both core graphs, over the same nine circuits as the H-grid suite. On the ring both routers complete all nine and the margin is  $-52.0\%$ ; read on the eight circuits TelesABRE completes on both architectures it is  $-53.8\%$  against the H-grid’s  $-49.1\%$ , so leaving the grid widens the gap rather than costing it. On the star it *narrows* to  $-41.5\%$ , tightest on QPEexact at 120 EPR against 132—the closest race dSABRE runs anywhere in this paper, and still a win. TelesABRE fails on Random, which dSABRE routes at 475 EPR.

2) *Large circuits*: To stress-test dSABRE beyond the main evaluation range we run it and TelesABRE on the MQT-Bench QFT at **100**, **200** and **360** logical qubits. Core size is held at  $5 \times 5$  and only the core count grows— $2 \times 3$ ,  $3 \times 4$ ,  $4 \times 5$  (6, 12 and 20 cores)—so logical qubits per core stay near-constant at 16.7–18.0 while the core graph lengthens, diameter  $3 \rightarrow 5 \rightarrow 7$ . Fixing occupancy is what makes the three rows one series rather than three unrelated points.

Table III reports them. dSABRE leads at both sizes where TelesABRE converges, and the margin widens with diameter: at 100q’s diameter-3 core graph there is less for the capacity term and lookahead to decide ( $-19.0\%$ ), while 200q’s diameter-5 graph gives those mechanisms real decisions to make ( $-37.8\%$ ). At **360q on twenty cores TelesABRE does not converge on any of three seeds**—every attempt exhausts its ten internal retries—while dSABRE completes at **1890 EPR**.

3) *Compile time*: All three compilers are timed on the 64-qubit suite, over the eight circuits TelesABRE completes. In geometric mean dSABRE leads at 1.9s against TelesABRE’s 4.7s and pytket-dqc’s 65s. Per circuit it splits four-four with TelesABRE but asymmetrically, losing by under a second and winning by up to  $9.9\times$ ; it beats pytket-dqc everywhere, by  $20\text{--}118\times$  above 1000 gates, that tool’s cost being set by hypergraph partitioning rather than by the gates routed. On three of the eight its figure is the PARTITION-HETEROGENEOUS fallback, timed with the abandoned embedding attempt included.

#### D. Ablation Study

We ablate dSABRE’s design choices along three axes: the scoring terms that price EPR pairs, the mechanisms that instead price feasibility, and continuous parameter sensitivity. All ablated configurations inherit the defaults of Table I and report the *geometric-mean EPR count* (gmEPR) over the whole 64-qubit suite, so every ablation figure below shares Table III’s basis.

1) *Mechanism ablation*: Table IV disables one component at a time on the 64q suite. That suite runs at  $\sim 67\%$  qubit utilisation. The rows are grouped by what they price: the scoring terms of Eq. 6, which trade EPR pairs against each other; capacity as a mechanism, where the legality floor and the score term  $c_{\text{pen}}$  do separate jobs and are therefore crossed; and the initial layout.

TABLE IV

ABLATION ON THE 64-QUBIT SUITE. ONE COMPONENT IS DISABLED PER ROW; ALL OTHERS STAY AT THEIR DEFAULTS, AND THE PROTOCOL AND HARDWARE BUDGET ARE THE MAIN RESULTS<sup>†</sup>. **FULL** IS TABLE III’S OWN GEOMETRIC MEAN OVER THE WHOLE NINE-CIRCUIT SUITE (173.3), NOT A SUBSET’S. *Neither* IS THE ONLY ROW THAT FAILS TO COMPLETE THE SUITE, SO IT HAS NO COMPARABLE WHOLE-SUITE MEAN; OVER THE SEVEN CIRCUITS EVERY ROW DOES COMPLETE IT COSTS  $+87.8\%$  AGAINST FULL’S 110.6.

| Configuration                                  | gmEPR | $\Delta$ (%)             | aborts   |
|------------------------------------------------|-------|--------------------------|----------|
| <b>Full</b> (baseline)                         | 173.3 | —                        | 0        |
| <i>Scoring terms</i> (Eq. 6)                   |       |                          |          |
| No capacity score term ( $c_{\text{pen}}=0$ )  | 189.5 | +9.4                     | 0        |
| Topological extended set (not BFS)             | 194.8 | +12.4                    | 0        |
| <i>Capacity as a mechanism</i> (Section III-F) |       |                          |          |
| No legality floor (score term only)            | 173.6 | +0.2                     | 0        |
| <b>Neither floor nor score term</b>            | —     | <b>+87.8<sup>†</sup></b> | <b>2</b> |
| Strict floor ( $\phi_1=r+1$ )                  | 201.5 | +16.3                    | 0        |
| <i>Initial layout</i> (Section III-G)          |       |                          |          |
| TelesABRE’s layout, not ours                   | 189.4 | +30.9 <sup>‡</sup>       | 0        |

<sup>†</sup> over the seven circuits every row completes; not comparable to the nine-circuit means above. <sup>‡</sup> against dSABRE’s own 144.7 over the eight circuits TelesABRE completes, the only ones for which its layout exists.

Substituting a topological-order extended-set construction costs 12.4%, and the gain grows with the depth of the remaining DAG, as Section III-E predicts.

**The two capacity mechanisms are substitutes, and together they are the largest effect in the table.** Removing either alone is close to free—+9.4% for the score term, +0.2% for the legality floor—while removing both costs +87.8% and aborts Multiplier and QNN outright.

**Only the hard condition makes completion unconditional.** The score term alone happens to avoid every abort on this suite, which is why the submitted design completed it, but it carries no guarantee that it will—and the two circuits *Neither* loses are exactly the two that router’s own capacity ablation lost. The legality floor’s contribution is therefore not in the EPR column at all, but in the abort column and Section III-F’s theorem. The row above it prices the alternative: making Tier 1 preserve the full reserve unaided costs +16.3% for no additional guarantee, which is why the default separates the two conditions.

2) *How often the guarantee is used*: The rows above price mechanisms by removing them; this one counts, on the shipped default, how often the completion guarantee of Section III-F is invoked. The *guarantee* is the all-or-nothing transaction: when scoring stalls, the router restores its last checkpoint and hands the stuck gate to a plan that walks both operands to a meeting core and executes there, which Eq. 8 makes always possible. One transaction retires one gate, so its count is also a gate count, and is read two ways: *reported* counts only the route whose EPR figure reaches Table III, *search* all nine routes behind it—three SabreLayout seeds by the three passes of Section III-G—of which the reported route is one, so search includes it. A *relay* is a different unit. To give a core the free slot Eq. 8 asks for, the router walks slack in from the nearest core that has some; each hop teleports one uninvolved

TABLE V

GATES RETIRED BY THE GUARANTEED TRANSACTION OF SECTION III-F, AND RELAY TELEPORTS, OVER EVERY SUITE IN THIS PAPER. THE TWO TRANSACTION COLUMNS ARE NESTED—SEARCH COVERS ALL NINE ROUTES PER CIRCUIT, OF WHICH THE REPORTED ONE IS A MEMBER—WHILE RELAYS ARE TELEPORTS, WHOLE-SEARCH, AND NOT TIED TO A TRANSACTION. THE COLUMNS ARE DEFINED IN THE TEXT.

| Suite        | Cores | Circuits  | Transactions |             | Relays<br>(search) |
|--------------|-------|-----------|--------------|-------------|--------------------|
|              |       |           | Reported     | Search      |                    |
| 25q          | 4     | 6         | 0            | 0           | 4                  |
| 36q          | 4     | 6         | 0            | 0           | 5                  |
| 64q          | 6     | 9         | 0            | 42          | 145                |
| 100q         | 6     | 2         | 6            | 73          | 250                |
| 200q         | 12    | 2         | 10           | 214         | 756                |
| 360q         | 20    | 2         | 66           | 745         | 3822               |
| <b>Total</b> |       | <b>27</b> | <b>82</b>    | <b>1074</b> | <b>4982</b>        |

qubit the other way—a *filler*, never an operand of the gate being routed—so that the empty slot advances, at the price of an EPR pair. Both the transaction and ordinary deadlock recovery relay, which is why relays appear on suites where no transaction fires, and they are counted whole-search only.

On the 21 circuits of the 25-, 36- and 64-qubit suites no reported route invokes the transaction, though at 64 qubits the surrounding search invokes it 42 times. Both counts grow with circuit and architecture size, reaching 66 reported and 745 across the search at 360 qubits, where 27 279 gates are routed in all. The transaction’s rollback paths are assertions on the theorem’s hypotheses—no donor core, no meeting core, a relay that cannot complete—so that a violated one would be visible rather than silent; across all 1074 none fired. Nor is the scored path any different: under those conditions the cheapest candidate provably always commits, and none of the 1.08 million candidates scored across four architectures was rejected.

3) *The initial layout*: The last row prices Section III-G’s layout by replacing it: dsABRE routing from TelesABRE’s own `optimize_initial` layout, so that only the routing loop differs. It costs 30.9%, and the margin over TelesABRE persists without our layout—33.4% against 49.1% with it.

4) *Parameter sensitivity*: Sweeping each continuous hyperparameter one-at-a-time, every default sits on a flat region of its curve (within  $\sim 2\%$  of the local minimum); only pushing a control far from its default causes a sharp regression, e.g.  $\gamma=1.0$  (flat weighting) costs 7% gmEPR. No single hyperparameter is a hidden lever. The full sweep is reported separately, with the cost model swept alongside it: raising  $c_{\text{tele}}$  from 10 to 100 at fixed  $c_{\text{swap}}$  widens dsABRE’s margin monotonically on every suite.

### E. Comparison with `pytket-dqc`

`pytket-dqc` is not a router but a hypergraph partitioner emitting gate teleportations. It reports its cost in *e-bits*, one e-bit being one shared EPR pair, so its counts are in our unit; we keep its term in this subsection, where its own numbers are read. Of the four differences listed in Section I, two are architectural assumptions rather than cost-model conventions,

TABLE VI

`PYTKET-DQC` AGAINST dsABRE, BY SUITE. *published* IS THE NETWORK IT IS NORMALLY GIVEN, *physical* THE ONE MATCHING dsABRE’S DEVICE, WHICH IS THE COLUMN READ IN THE TEXT. SUITE FIGURES ARE GEOMETRIC MEANS OVER THE CIRCUITS OF TABLE III.

| Suite             | pytket-dqc e-bits |          | dsABRE |              |         |
|-------------------|-------------------|----------|--------|--------------|---------|
|                   | published         | physical | EPR    | $\Delta$ (%) | unreal. |
| 25 logical qubits | 35.3              | 27.5     | 15.3   | −44.4        | 4/6     |
| 36 logical qubits | 12.3              | 10.6     | 10.5   | −0.9         | 2/6     |
| 64 logical qubits | 168.0             | 176.3    | 173.3  | −1.7         | 7/9     |

and both can be corrected; Table VI reports the comparison before and after, throughout using its strongest distributor, COVEREMBEDDINGSTEINERDETACHED, best of five seeds.

*Correcting the network*. The network `pytket-dqc` normally given is more permissive than our device in two respects. Its per-server *communication capacity*—the number of link qubits a core may hold simultaneously—is unbounded unless set, whereas a core here has one port per incident link,  $\deg(c) = 2$  or 3; and its server sizes split the *logical* qubit count evenly, whereas a core physically offers  $M - \deg(c)$  data slots.

*The corrected counts are still not executable*. Communication capacity does not reach the cost model: as noted in [19], none of the distribution routines consider it, and declaring it left all 21 distributions unchanged. Generating the circuit is what tests executability, and usually it fails: **13 of the 21 distributions cannot be built within the communication capacity the device provides**, AE at 25 qubits needing 19 simultaneous link qubits per core against the 2 available. This is no coincidence—a hyperedge over  $g$  gates buys a factor of  $g$  in e-bits and pays in link-qubit occupancy, so the cheapest-looking distributions demand the most memory—and the repair pass that would restore executability does not reach them: on the 25-qubit suite it returns only on the two distributions that already fit, and not within 300 s on any that do not.

*What survives the corrections*. Read with that caveat in force—most `pytket-dqc` counts below are the cost of a distribution its own tool cannot build on this device (Table VI, last column)—the direction is set by circuit structure rather than size. dsABRE wins five of six circuits at 25 qubits; at 64 the comparison reverses on four of nine—QPEexact, where Steiner-tree embedding reuses a few pairs across many controlled operations (+58.5%), AE (+16.2%), QAOA (+17.3%), and the trivially small Graphstate (7 against 15 e-bits, a large percentage on a circuit too small to matter)—while dsABRE leads on the densest circuits (Multiplier −55.6%, QNN −10.9%).

*Entanglement lifetime*. A second assumption is separable from the first: capacity is how many pairs are live at once, lifetime is how long one must live. Every e-bit count above assumes an *unbounded* lifetime, a link qubit staying entangled until the last gate of its hyperedge. Bandini *et al.* [20] argue this is unrealistic and cap gates-per-EPR at  $D_{\text{max}}$ , proposing  $D_{\text{max}}=3$ . Re-scored under that bound by a counter that generalises `pytket-dqc`’s own cost model and reproduces

it at  $D_{\max}=\infty$ —dsABRE needs no adjustment, a teledata pair being consumed instantly—the `pytket-dqc` lead (+15.0% at  $D_{\max}=\infty$ ) survives only above a threshold: the geometric-mean gap crosses parity between  $D_{\max}=\infty$  and 10, and becomes a dsABRE win of 46.5% at  $D_{\max}=3$ . Gate teleportation’s e-bit advantage is therefore conditional on sustaining entanglement across  $\gtrsim 10$  gates per pair; teledata routing delivers its counts under any lifetime. Per-circuit sweeps are in the online appendices.

## V. RELATED WORK

The Introduction already places dsABRE within the partition–communicate–schedule decomposition of the DQC compilation stack and lists the major lines of work in each stage. Here we focus on what specifically distinguishes dsABRE from the closest prior art.

**SABRE-style distributed routers.** TelesABRE [12] is our closest prior work and direct experimental baseline. Section III-A states what dsABRE scores differently; the three differences that produce the measured gap are where it prices *availability* (a macro-action against a bare hop that an occupied port deletes), how it *selects* (a priority rule against a pooled argmin with a flat bonus, under which 59.5% of TelesABRE’s 64-qubit communication operations fire while a same-core front gate is still waiting), and how it charges *capacity* (graded in the landing core’s slack and backed by a legality floor, against a fixed penalty charged along the whole Dijkstra path). TelesABRE supports telegate candidates while dsABRE currently uses teledata only.

DMapS [13] is a concurrent end-to-end DQC compiler pairing a KaHyPar partitioner with a SABRE-derived router, and shares our joint EPR-plus-local-SWAP objective. Its only cross-core primitive is a two-EPR *remote SWAP*, where dsABRE uses one-way teledata into a layout-reserved free slot. Over the whole nine-circuit 64-qubit suite that asymmetry shows: dsABRE uses  $1.43\times$  fewer EPRs in geometric mean and  $1.5\text{--}3.1\times$  fewer on each of the seven circuits above 1000 CX. DMapS wins only on GHZ and Graphstate, which KaHyPar partitions almost perfectly.

Allocation methods built on hypergraph partitioning [3], [4] are complementary to dsABRE: any can supply the initial layout its routing loop consumes. Similarly, time-aware schedulers [7], [8] sit upstream or downstream of the routing stage dsABRE addresses.

## VI. FUTURE WORK

**Composing dsABRE with burst execution and telegate.** dsABRE optimises EPR count on a fixed circuit DAG using teledata alone (Section III); burst execution and telegate are two views of the same next step, not two separate ones. Burst-execution passes such as AutoComm [21] and its collective extension QuComm [22] operate earlier in the toolchain: they use gate commutativity to reorder consecutive non-local gates on the same wire (or across multiple wires) and amortise the whole block onto a single shared entanglement state, reporting EPR savings of 50–75% on amenable circuits, which the lifetime-bounded grouping of Bandini *et al.* [20] tempers with

the  $D_{\max}$  constraint of Section IV-E. That amortisation is exactly what a telegate candidate needs to be worth scoring: Section III identifies the obstacle as a *grouping* decision, and a burst-extraction pre-pass is one way to supply it externally, exposing wire-aligned sequences for dsABRE’s BFS extended set to score as a unit. It is also where the reversal of Section IV-E comes from—gate teleportation beats dsABRE on QPEexact and AE, whose long shared-control sequences such a pre-pass would expose—so closing that gap and adding telegate are one open question, not two. Quantifying the joint improvement—especially on circuits where a burst’s beneficiary core is itself a function of the routing decisions dsABRE makes—remains open.

**Parallel teleportation for circuit-runtime minimisation.** dsABRE optimises EPR count and emits one teleport per iteration; wall-clock runtime depends additionally on whether teleports with disjoint links and staging sets can fire concurrently. A natural follow-on is a scheduling pass over dsABRE’s teleport trace that batches non-conflicting moves, co-designed with denser inter-core links [23] and entanglement buffer qubits [22], [24]. Time-sliced compilers [7], [8] provide a starting point but are not paired with a SABRE-style scoring loop; naively adding links without retuning the routing energy can *increase* EPR consumption.

**Beyond EPR count: latency and fidelity.** This paper optimises a resource count, not an execution time or an output fidelity. Two of the three system-level costs are already partly visible: intra-core SWAP count, the dominant contributor to routed depth, is reported beside EPR count in every table and falls alongside it, and entanglement quality enters through the lifetime bound of Section IV-E. A joint objective is the same scheduling layer as the parallel-teleportation pass above, asked to do more: EPR count, depth and per-operation error rates are not interchangeable—a teleport is slow and noisy, a SWAP fast and cheap, and the exchange rate is hardware-specific—so it needs a device error model on top of the conflict check that concurrency alone requires. The cost-ratio sweep is a first step, the conclusions holding across the two orders of magnitude of relative teleport cost separating microwave-coupled from photonic-link hardware.

## VII. CONCLUSION

We presented dsABRE, a SABRE-style router for multi-core distributed quantum computers. It carries SABRE’s objective across cores under a single rule—the resource a move spends, plus the capacity it consumes, plus the progress it buys—whose two added terms are not independent heuristics but the two respects in which a teleport is unlike a SWAP: it fires only at a communication port, and it consumes a free slot in the core it lands in (Section III-A). Across 21 MQT Bench circuits at 25–64 logical qubits that rule cuts geometric-mean EPR consumption by 49–62% against TelesABRE, and it scales to 360 logical qubits on twenty cores. Capacity handling is what carries it: ablated jointly with the legality floor that substitutes for it, capacity is worth 87.8% and two completions, against 12.4% for the BFS-layer extended set. A capacity condition checked once before routing turns that from



a tendency into a guarantee—dsABRE routes all four circuits here on which TelesABRE does not converge, and under five checked preconditions termination and success coincide by construction (Section III-F). More broadly, the results indicate that distributed routing benefits less from a richer per-candidate formula than from letting the architecture’s capacity signal reach the scorer in *graded* form—and, independently of any score, from never leaving feasibility to a heuristic’s preference.

*Artefact availability.*: The dsABRE implementation, the routed-circuit benchmark suites, per-circuit JSON results, and the supplementary material are available at <https://github.com/ebyon72/dsabre>.

## ACKNOWLEDGMENTS

This work was partially supported by the Australian Research Council (Grant No. DP250102952 and DP220102059). The author used Anthropic’s Claude (via Claude Code) to assist with refactoring parts of the implementation, authoring benchmark scripts, drafting figure sources, and copy-editing. All AI-generated content was reviewed and verified by the author, who takes full responsibility for the correctness and originality of the work.

## REFERENCES

- [1] D. Cuomo, M. Caleffi, and A. S. Cacciapuoti, “Towards a distributed quantum computing ecosystem,” *IET Quantum Communication*, vol. 1, no. 1, pp. 3–8, 2020.
- [2] P. Escofet, A. Ovide, M. Bandic, L. Prielinger, C. G. Almudever, S. Feld, E. Alarcón, and S. Abadal, “Revisiting the mapping of quantum circuits: Entering the multi-core era,” *ACM Transactions on Quantum Computing*, 2024, arXiv:2403.17205.
- [3] P. Andres-Martinez and C. Heunen, “Automated distribution of quantum circuits via hypergraph partitioning,” *Physical Review A*, vol. 100, no. 3, p. 032308, 2019.
- [4] P. Andres-Martinez, D. Mills, T. Forrer, and L. Henaut, “CQCL/pytket-dqc,” <https://github.com/CQCL/pytket-dqc>, Jun. 2024.
- [5] C. H. Bennett, G. Brassard, C. Crépeau, R. Jozsa, A. Peres, and W. K. Wootters, “Teleporting an unknown quantum state via dual classical and Einstein-Podolsky-Rosen channels,” *Physical Review Letters*, vol. 70, no. 13, pp. 1895–1899, 1993.
- [6] J. Eisert, K. Jacobs, P. Papadopoulos, and M. B. Plenio, “Optimal local implementation of nonlocal quantum gates,” *Physical Review A*, vol. 62, no. 5, p. 052317, 2000.
- [7] D. Ferrari, A. S. Cacciapuoti, M. Amoretti, and M. Caleffi, “Compiler design for distributed quantum computing,” *IEEE Transactions on Quantum Engineering*, vol. 2, pp. 1–20, 2021.
- [8] J. M. Baker, C. Duckering, A. Hoover, and F. T. Chong, “Time-sliced quantum circuit partitioning for modular architectures,” in *Proceedings of the 17th ACM International Conference on Computing Frontiers (CF)*, 2020, pp. 98–107.
- [9] L. J. Stephenson, D. P. Nadlinger, B. C. Nichol, S. An, P. Drmota, T. G. Ballance, K. Thirumalai, J. F. Goodwin, D. M. Lucas, and C. J. Ballance, “High-rate, high-fidelity entanglement of qubits across an elementary quantum network,” *Physical Review Letters*, vol. 124, no. 11, p. 110501, 2020.
- [10] S. Daiss, S. Langenfeld, S. Welte, E. Distant, P. Thomas, L. Hartung, O. Morin, and G. Rempe, “A quantum-logic gate between distant quantum-network modules,” *Science*, vol. 371, no. 6529, pp. 614–617, 2021.
- [11] G. Li, Y. Ding, and Y. Xie, “Tackling the qubit mapping problem for NISQ-era quantum devices,” in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 1001–1014.
- [12] E. Russo, E. Vinciguerra, M. Palesi, D. Patti, G. Ascia, and V. Catania, “TeleSABRE: Heuristic layout synthesis in multi-core quantum systems with teleport interconnect,” in *Proceedings of the IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2025, pp. 749–758, arXiv:2505.08928. Code: <https://github.com/Haimrich/telesabre>.
- [13] T. Luo, Y. Zheng, Y. Deng, and X. Fu, “DMapS: End-to-end qubit mapping and routing for distributed quantum computing architectures,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2025, code: <https://github.com/RoccoLoter/DMapS>.
- [14] A. Barenco, C. H. Bennett, R. Cleve, D. P. DiVincenzo, N. Margolus, P. Shor, T. Sleator, J. A. Smolin, and H. Weinfurter, “Elementary gates for quantum computation,” *Physical Review A*, vol. 52, no. 5, pp. 3457–3467, 1995.
- [15] Qiskit contributors, “Qiskit: An open-source framework for quantum computing,” 2024.
- [16] H. Zou, M. Treinish, K. Hartman, A. Ivrii, and J. Lishman, “LightSABRE: A lightweight and enhanced SABRE algorithm,” 2024. [Online]. Available: <https://arxiv.org/abs/2409.08368>
- [17] N. Quetschlich, L. Burgholzer, and R. Wille, “MQT Bench: Benchmarking software and design automation tools for quantum computing,” *Quantum*, vol. 7, p. 1062, 2023.
- [18] C. Chamberland, G. Zhu, T. J. Yoder, J. B. Hertzberg, and A. W. Cross, “Topological and subsystem codes on low-degree graphs with flag qubits,” *Physical Review X*, vol. 10, no. 1, p. 011022, 2020.
- [19] P. Andres-Martinez, T. Forrer, D. Mills, J.-Y. Wu, L. Henaut, K. Yamamoto, M. Murao, and R. Duncan, “Distributing circuits over heterogeneous, modular quantum computing network architectures,” *Quantum Science and Technology*, vol. 9, 2024, arXiv:2305.14148.
- [20] M. Bandini, D. Ferrari, S. Carretta, and M. Amoretti, “Optimized compilation for distributed quantum computing,” 2026, arXiv:2602.24062.
- [21] A. Wu, H. Zhang, G. Li, A. Shabani, Y. Xie, and Y. Ding, “AutoComm: A framework for enabling efficient communication in distributed quantum programs,” in *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, arXiv:2207.11674.
- [22] A. Wu, Y. Ding, and A. Li, “QuComm: Optimizing collective communication for distributed quantum computing,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2023.
- [23] P. Escofet, S. B. Rached, S. Rodrigo, C. G. Almudever, E. Alarcón, and S. Abadal, “Interconnect fabrics for multi-core quantum processors: A context analysis,” in *Proceedings of the 16th International Workshop on Network on Chip Architectures (NoCArc)*, 2023, arXiv:2309.07313.
- [24] J. Liu, A. Zang, M. Suchara, T. Zhong, and P. D. Hovland, “Hardware-software co-design for distributed quantum computing,” in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, 2025, pp. 1–6.



**Sanjiang Li** received his B.Sc. in mathematics from Shaanxi Normal University in 1996 and his Ph.D. in mathematics from Sichuan University in 2001. He is currently a professor at the Centre for Quantum Software and Information at the University of Technology Sydney (UTS). Prior to joining UTS, he worked in the Department of Computer Science and Technology at Tsinghua University from 2001 to 2008. His primary research interests include knowledge representation, artificial intelligence, and quantum circuit compilation.